P1 ( ) — Generalization as Formalized Induction

: v1.0 | : 2026-08-11 : 10 + Appendix
_P1_ .md (§1.0-1.7) .md : / , (;
) : (=) [ ]

Title ( )

Generalization as Formalized Induction: Token-Native Factorized Representation and Weight Compilation in a Minimal Transformer

Abstract ( )

We propose that systematic generalization is not an emergent or a priori faculty of neural networks, but the formalization of induction. To generalize on a concept A, A must be precisely defined, and every concept in A's definition chain must be sufficiently trained; sufficient training is a relational property — parallel concepts under the same superordinate concept, wrong-answer examples, and collocation coverage — not a quantitative one. We operationalize this in a token-native representation (B/C/S/G/P/A projection layers) in which conceptual degrees of freedom are factorized according to their symmetries, and execution is driven by token definitions (a zero-hardcoding evaluator). A 2-layer, <1 MB transformer trained on ~2000 samples in a few epochs generalizes to arbitrary width (2000-digit addition, acc 1.000), unseen radices ([3..9], OOD 0.998), unseen operator orders (iteration chain), and joint Cartesian OOD. Symbol permutation invariance holds (0.996→0.987 under 26080 digit renamings), and the three execution channels — construction, intuition, formal rewriting — are semantically equivalent (1.000 on 2000-digit carry chains), the neural implementation of Computational Trinitarianism. We emphasize that intuition is the reasoning fast path: it is hard to obtain (acquisition requires config-dependent epochs — 3 for simple, 8 for the full logic+arithmetic suite), but once formed it generalizes easily by skipping steps — jumping from input to output over the intermediate construction steps. Acquisition and step-skipping generalization are decoupled.

---

1. Introduction

1.1

1. , Compositional generalization (Lake et al., 2017; Keysers et al., 2020 COGS; SCAN) , ” ”, ” ”

2. **primitive,** -calculus ” → tokenize → transformer” (application/abstraction/ -reduction) primitive : Church , — ( ) , / /
3. :
   - “ / ” ( ) — , : token (E0/E61);
   - “ benchmark engineering / + ” — : , (, SGD)
4. / (NALU, Neural Arithmetic Logic Units): , ; ” / ”
5. : (2 /<1MB/2000 /3epoch) , , / *representation engineering* ,

1.2 Babel

Babel token , : — A A, A token :

- : (base × direction × inverse × level × domain), (B /C /S /G /P /A) token
- : ( ), / token ; ( )
- : = (+ +), trace —
- : (/ / /), (0.987), (1.000), ( )

1.3 (5 , =)

- **C1** — = **(Formalized Induction)**: / , : (G1) + (G2) (E0, E61)
- **C2** — **token** : (B/C/S/G/P/A) + + , ” ” (/ /),
- **C3** — : 2 <1MB ~2000 , (2000) / unseen radix / unseen order / OOD; (0.987) + (2000 / 60/100% / 1.000)
- **C4** — **(Weight Compilation)**: = ( ) ; answer trace, — : ( ), ( )
- **C5** — **(Three-Channel Equivalence)**: / / (exp50: 2000 1.000) — Computational Trinitarianism (Curry-Howard) ,

---

2. Problem Setting

2.1 systematic generalization

- ( ): : × × × ×
- ” ” : (/ / /),

2.2 / OOD

- **(in-distribution)**:

- **OOD (out-of-distribution)**: (; ;) (—)
- : acc ( ); acc (token) — ”digit_nine 0.16 vs 1.000” ,

2.3 Babel grammar

- token (G gtoken + P), : arg/fn/args/binder/body : atom / application / equality / binary_connective / unary_connective / quantified / eos
- numeral: [sign +] + digit (× digit), (digit)
- prop: (judgment), gtoken/ptoken ,
- : (/ /neg), (9), , , (coercion)

---

3. Babel ( )

3.1 (token ” ”, json)

B	baseloop.jsonl	axiomatic , (brace)
C	derive.jsonl	5 (explicit/inductive/recursive/axiomatic/implicit)
S	symbol_tokens.jsonl	(glyph → maps_to , ,)
G	gtokens.jsonl	(/), token
P	presentation.jsonl	(/ /),
A	arrow_tokens.jsonl	(/ / /coercion)

: token ; token/ , ( )

3.2

= , token: A = (d1, …, dk) : - : (succ→+→×→^→↑↑) — ” ” - : inverse (succ/pred, power/root, +/−) — ” ” - : natural/integer/rational/real/complex — ”domain” - : sign/base/cardinality — ” ”

3.3

- : iterate(, ,) — ; F_{n+1}(a,b) = Iterate(F_n, a, b)
- : inverse (power root), (neg/scale/root; complement/parallel_sum De Morgan ; translation/inversion)
- : coercion (A concept=coercion ,)

3.4 composition

- G (gtoken), ; SKI gtoken (reduction)
- : → arrange → gtoken → AST → (, parse/print)

3.5 iteration

- iterate : [, fn, ,] A (ARROW_REGISTRY), is_succ
- : , (translation/inversion/differential) — token token , eid ,

3.6 domain/radix representation

- : numeral = [sign, base, cardinality] digit ; base (base ,)
- (fraction/radical/imaginary_unit), numeral (digit), " value" digit
- : ( ) vs [ ] — = vs (, E4.1)

---

4. Weight Compilation

4.1

- : = (synth_core,) : balanced/trace/numeral_split/fill/choose/definition/arrow/law/logic
- (G2): ( ) + (,) + (3,)
- : gtoken/ptoken (assemble_seq/judge_seq), = " token"

4.2 Python reference evaluator

- (eval/engine.py): eval_op(op, arg_tokens) → result_tokens, token token
- token : / / (logic_eval/compare_eval/digit_eval/arith_eval/symmetry_eval/prime_eva
- : (token_iterator) ( ) →

4.3 Transformer

- : 2 transformer, dim 64, <1MB
- : CE ( ) + CE ( )
- : ~2000 , epoch (3 epochs, + 8 epochs — exp53) , (EXP-80)
- = ( ),
- : per_token_gen token / epoch_gen / crash_culprit /

4.4 input answer trace

- (/), **trace** " " " " : trace, ( ); trace,
  —
- : = (, oracle) → → (,)
- : , (weights program, tokens tape)

---

5. Experiments

run_exp (: config/metrics/model/samples/views) : acc / acc [ ] =

	( )
: known operators (8+ /)	arith_multi_type 1.000 (1188)
2000-digit addition	acc 1.000
unseen radix ([3..9] × 20)	OOD 0.998
unseen order ( )	
joint Cartesian OOD (× × ×)	[ ] (0.816 / 0.988) /
(exp41)	26080 digit , 0.996→0.987 ( )
(exp50)	= (2/9/20/2000), 1.000; 2000
(exp51)	2000 : 0.62s / 0.086s (7×) / 0.069s (9×)
(exp20)	iff/xor → 0.798 (1.0→0.833); and/or → 0.814
(exp53)	epoch 1/2/3/5/8 → 0.001/0.214/0.244/0.781/0.996 — SUPPORT: , (3 / 8)
(exp80)	2000 / 60 / 100% digit / × **1.000** (8 epochs)
(exp90,)	20 : 82 token/0.410ms vs 5 token/0.186ms (num , 16× token , 2.2×), ; = (itoken vocab)

	( )
(exp60)	, — G6
	( )
imply	+ + 0.996
(exp10_imply_supervised)	(imply 9)
anonymous operators	[ ]
(operator)	
/ /epoch	2 / 64 dim / <1MB /
	~2000 / 3 epochs

5.x

- **known operators**: / /neg + , ,
- **2000-digit**: L, 2000 — , ” ”
- **unseen radix**: base , — ” ”
- **unseen domain/order**: — ( ),
- **joint OOD**: ,
- **imply (exp10_imply_supervised)**: + + imply 9 (0.996) imply (exp20 imply 0) — ” , + ”
- **anonymous operators**: digit/operator , — (exp41 0.987 / exp80 100% 1.000)

---

6. (Semantic-Path Compilation)

6.1 0

- : K0 , T_ K0(T_ , x) = y — ( )
- : ; 0 (exp50: 2000)

6.2 — , ( )

- ( ): = (+ , epoch — exp53 8 epochs), — ” ” , ;
- : , Sem_fast( ) = Sem_expanded(T_) (/fallback); Cost_fast < Cost_expanded (exp51: 2000 7×)
- — ,
- : teacher student / ( ); fallback
- : (eval_op, 0.069s) (0.086s) — (, oracle) + (exp41 0.987 / exp50 2000 1.000),

---

7. Ablations

Ablation		
(numeral_v4) vs / (wrong-symmetry/ablated)		G4/C1
(exp20)	iff/xor → 0.798, 0.000, 1.0→0.833; and/or → 0.814	G2
(exp41)	26080 digit 0.996→0.987 ( )	G4
(exp50)	= = (2000 1.000)	R/C5
(exp53)	1/2/3/5/8 → 0.001/0.214/0.244/0.781/0.996 — SUPPORT:	G5
(exp80)	2000 / 60/100% / 1.000, 8 epochs — ,	G5/R
(exp60)	, — ( )	
(neg 9→15)	neg 0.000→0.958	G2
token (value_three =1)	,	G1
(vocab 153)	40 epochs	G0/G6
(( ) vs [ ]) vs (focused)	0.904 → 0.000 ( )	G4
representation baseline (EXP-C1: radix-fixed / iteration-hidden / entangled)	[ ]	C1

baseline (EXP-C1) : / , (/ /) (0.5); (exp02) (1.000)

(§7.2): EXP-10 (0-IMPLY) bug — " " + : ( ) ; branch run_exp ; (imply + , exp10_imply_supervised 0.996) ,

---

8. Related Work

1. **-calculus** — : primitive, ; Church " ": / /
2. — Lake et al. 2017; Keysers et al. 2020 (COGS); SCAN; : ; (/)
3. — NALU/NAU ( ), Neural Arithmetic: ; +

4. — / : ;
5. — disentangled/compositional structured representations: ” ”; ” ”,
6. → — Learning to Compile Programs to Neural Networks (ICML 2024) : ; = , , +fallback
7. — Frege (Sinn/Bedeutung), Saussure (/ / /): G4 ; Kahneman (System 1/2): G5
8. — — (de Bruijn/OpenMath/ /Bourbaki/ /Mizar-Lean-Isabelle), **claim**; claim (/ /) = Computational Trinitarianism : (= ,)
9. — Fodor (language of thought), Chomsky ( ): ; E0/E61

---

9. Limitations

1. **serialization ( )**: — (token) , ; AR
2. **finite resources**: / /epoch ; epoch (3, 8 — exp53); , + max_val ( )
3. **supported calculus** : ( ); ;
4. / : bool , (/) , numeral_value
5. : , — /fallback ( )

---

10. Conclusion

- = : + , ,
- : — ,
- : = ; trace, = : , —
- : / / (exp50: 2000) —
- : , ( ), ( )

Appendix

- A. : config/seed/ /per_token /conclusion (docs/paper_data/results.csv)
- B. Grammar: gtoken/ptoken ( ); (add_infix/and_infix/not_prefix/succ_prefix)
- C. examples: (numeral_split/law/ /)
- D. Training config: JSON (exp02 : synth.samples + judge + train)
- E. Lean proofs: (/) + — , claim

: (→)

- ☐ EXP-41 (anonymous operators) — 5
- ☐ EXP-50 — C5
- ☐ EXP-51 — 6
- ☐ representation baseline (7)
- ☐ ([ ]) + paper_meta
- ☐ (Related Work)
- ☐ Lean

(, 2026-08-11)

, , :

- (→ : token ; attention);
- **CoT flashing** (→ : ; synthetic CoT);
- (→ : , ; distilled intuition / flashing).

, — ASI ,